Surplus + ATLAS: instalación desde cero

Punto de partida: solo recibiste `surplus_matching_private.mpc.zip`. Este ZIP contiene el programa MPC, no contiene MP-SPDZ, webSPDZ, Emscripten ni los inputs. Sigue estos pasos en macOS.

1. Instalar herramientas básicas

Instala Xcode Command Line Tools y Homebrew (si aún no los tienes). Después instala las herramientas de compilación:

```
xcode-select --install
# Instala Homebrew desde https://brew.sh si hace falta
brew install git cmake make python node openssl boost
```

Las dependencias concretas pueden variar según la revisión de webSPDZ. Si su README pide otras bibliotecas, instálalas también.

2. Descargar webSPDZ (incluye la base MP-SPDZ)

Clona la misma revisión de webSPDZ que utilizó tu compañera. Confirma con ella el repositorio y el commit exactos; como referencia, su proyecto anterior apuntaba a este repositorio:

```
cd ~
git clone https://github.com/tbblunch/webSPDZ.git
cd webSPDZ
# Si te comparten un commit específico:
# git checkout <COMMIT>
```

No necesitas instalar MP-SPDZ por separado si el repositorio webSPDZ que clonaste ya contiene `compile.py`, `Compiler/` y `Programs/`. Compruébalo:

```
ls compile.py Compiler Programs
```

3. Instalar y activar Emscripten

```
cd ~
git clone https://github.com/emscripten-core/emsdk.git
cd emsdk
./emsdk install <VERSION>
./emsdk activate <VERSION>
source ~/emsdk/emsdk_env.sh
emcc --version && node --version
```

Sustituye por la versión que usó tu compañera: en su Mac puede obtenerla con «cd ~/emsdk; ./emsdk list --installed». No adivines la versión. Al activar emsdk también puede cambiar la versión de Node utilizada.

4. Extraer SOLO el código recibido

Mueve el ZIP recibido a Descargas. Descomprímelo y copia el archivo .mpc a la carpeta de fuentes de webSPDZ:

```
cd ~/webSPDZ
unzip ~/Downloads/surplus_matching_private.mpc.zip \
  surplus_matching_private.mpc -d Programs/Source
ls Programs/Source/surplus_matching_private.mpc
```

5. Crear manualmente los inputs privados

El archivo adjunto lee **4 enteros por participante** (4 skills). El mínimo ideal **T = 10** ya está fijado dentro del programa: NO se introduce como quinto input. Los valores deben escribirse en el mismo orden de skills para todos los países.

Ejemplo de prueba editable. Cada línea es una skill, en orden de la 1 a la 4:

```
mkdir -p Player-Data
printf '4\n13\n6\n10\n' > Player-Data/Input-P0-0
printf '15\n3\n12\n7\n' > Player-Data/Input-P1-0
printf '8\n17\n5\n11\n' > Player-Data/Input-P2-0
printf '12\n8\n14\n4\n' > Player-Data/Input-P3-0
```

Estos números son ilustrativos, NO son los datos originales de la prueba de tu compañera y pueden producir otro emparejamiento. Para editar un país: nano Player-Data/Input-P0-0; sustituye las cuatro líneas, guarda con Ctrl+O, Enter y Ctrl+X. En esta demo los archivos se preparan antes de iniciar las pestañas; no hay formulario de entrada en la página.

6. Compilar el programa MPC

```
cd ~/webSPDZ
./compile.py surplus_matching_private
ls -lh Programs/Bytecode/surplus_matching_private*
ls -lh Programs/Schedules/surplus_matching_private*
```

El bytecode habitual se llama `surplus_matching_private-0.bc`, no `surplus_matching_private.bc`. Comprueba que exista antes de seguir.

7. Compilar ATLAS para el navegador

```
source ~/emsdk/emsdk_env.sh
cd ~/webSPDZ
make -B atlas-party.x -j2
ls -lh atlas-party.html
```

Hazlo **después** de compilar Surplus y crear los inputs, porque la construcción empaqueta las carpetas `Programs` y `Player-Data`. Si cambias los inputs o recompilas el programa, vuelve a ejecutar este paso antes de recargar Chrome.

8. Iniciar los servidores (dos terminales)

Terminal A — HTTPS:

```
cd ~/webSPDZ
python3 https-server.py localhost 8000
```

Terminal B — señalización WebRTC:

```
source ~/emsdk/emsdk_env.sh
cd ~/webSPDZ
node signaling_wss_server.js localhost 8080
```

Deja ambas terminales abiertas. La segunda debe mostrar «Starting Signaling Server at localhost:8080». En Chrome acepta el certificado local si se te solicita.

9. Abrir las cuatro pestañas de Chrome

Abre estas URL, una por participante, en cuatro pestañas del mismo equipo:

```
P0: https://localhost:8000/atlas-party.html?
arguments=-N,4,-T,1,-w,0,0,surplus_matching_private,-OF,.

P1: https://localhost:8000/atlas-party.html?
arguments=-N,4,-T,1,-w,0,1,surplus_matching_private,-OF,.

P2: https://localhost:8000/atlas-party.html?
arguments=-N,4,-T,1,-w,0,2,surplus_matching_private,-OF,.

P3: https://localhost:8000/atlas-party.html?
arguments=-N,4,-T,1,-w,0,3,surplus_matching_private,-OF,.
```

Cada URL está dividida aquí en dos líneas para facilitar la lectura: al pegarla en Chrome, elimina el salto de línea. No cierres ninguna pestaña hasta que termine el cálculo.

10. Comprobar el resultado y repetir

El código del ZIP revela a cada país únicamente su partner y muestra `YOUR PARTNER CODE = ...`. El valor -1 significa que no se encontró una pareja útil. La ejecución debe terminar con «Finished». La captura anterior de tu compañera mostraba un texto ligeramente distinto («YOUR RECOMMENDED PARTNER CODE»), por lo que ese resultado corresponde a otra revisión del archivo: para este ZIP, usa el texto que aparece en su código.

Para cambiar el experimento: detén las cuatro pestañas, modifica los cuatro archivos `Input-P*-0`, vuelve a compilar ATLAS (paso 7) y abre de nuevo las cuatro pestañas. No hace falta recompilar el .mpc si solo cambian los datos.

Nota: los cuatro archivos locales son cómodos para una demo en un solo equipo, pero no aíslan los datos entre usuarios independientes. Para una prueba con cuatro computadoras, cada participante debería conservar sus propios datos y habría que adaptar las direcciones de los servidores y la distribución de archivos.